

Processor Verification

Lab 7 - UVM Testbench

Lorenzo Pattaro Zonta

May 7, 2026

1 Introduction

2 Design Under Test

The design under test (DUT) is a simple memory controller that interfaces with two masters: a CPU and a DMA. The controller manages read and write requests from both masters and arbitrates access to a shared memory. The memory is organized as an array of 256 words, each 32 bits wide. The controller receives address, data, and control signals from both masters and generates appropriate responses based on the requests. The controller also handles reset and clock signals to ensure proper synchronization of operations.

Signal	Direction	Width	Description
addr	Input	8	Base address of transaction (word address)
wdata	Input	32	Write data (valid when we=1)
rdata	Output	32	Read data (valid when we=0)
we	Input	1	Write enable (1=write, 0=read)
req	Input	1	Request valid
gnt	Output	1	Master has been granted, request to be processed

Table 1: Memory Controller I/O Interface Signals

2.1 Behavioral description

The memory controller implements a dual-master arbitration scheme with fixed priority. The controller manages concurrent access to a shared 256-word $\times$ 32-bit memory from two independent masters: Master 0 (CPU) and Master 1 (DMA).

2.1.1 Arbitration Logic

The controller uses a fixed-priority arbitration scheme:

- **Single Master Valid:** When only one master issues a request ($req=1$), that master is serviced immediately by asserting its corresponding grant signal ($gnt=1$).
- **Both Masters Valid:** When both masters issue requests simultaneously, Master 0 (CPU) has priority and is serviced. Master 0's gnt signal is asserted while Master 1's gnt remains deasserted. Master 1 must wait until Master 0 deasserts its request before being granted access.
- **No Master Valid:** When no master issues a request, the controller remains idle with both grant signals deasserted.

2.1.2 Read and Write Operations

Read Operations ($we=0$):

- Reads are combinational with zero clock cycle latency
- Once address and grant signals are stable, read data is immediately available on the `rdata` output
- No memory update occurs

Write Operations ($we=1$):

- Writes are synchronous, occurring on the rising edge of the clock
- Memory update happens on the next clock cycle when both `gnt` and `we` are asserted
- Data at the specified address (`addr`) is updated with the value on `wdata`
- One clock cycle latency from request to memory update

2.1.3 Memory Organization

The internal memory consists of $2^8 = 256$ words, each 32 bits wide, providing $8 \times 256 = 2048$ bits of total storage.

3 Testbench Conversion to UVM

The System Verilog testbench will be rewritten following the UVM (Universal Verification Methodology) standard, decomposing the single module into reusable, hierarchical components. The conversion maintains the original test stimulus and checking logic while reorganizing them into a modular and standardized framework.

3.1 Overview of Changes

The primary transformation involves converting procedural tasks and mailbox-based communication into UVM classes and TLM (Transaction-Level Modeling) interfaces. The key changes are:

- **Transaction Definition:** The original packed struct `trx_t` is now encapsulated in the `controller_transaction` UVM class, extending `uvm_sequence_item`. This class includes proper UVM methods (`copy()`, `compare()`, `convert2string()`) for transaction handling.
- **Generator → Sequencer + Sequence:** The procedural `tb_generator()` task is replaced by a `controller_sequencer` (manages transaction flow) and `controller_sequence` (generates test scenarios). This decouples stimulus generation from application.
- **Driver:** The `tb_driver()` task becomes the `controller_driver` class, which responds to the sequencer via the standard UVM pull interface, applying transactions to DUT ports.
- **Monitor:** The `tb_monitor()` task is converted to the `controller_monitor` class, which samples DUT signals and broadcasts transaction events via TLM analysis ports.
- **Checker → Scoreboard:** The `tb_checker()` task is restructured as the `controller_scoreboard`, which receives predicted and actual transactions via TLM ports and maintains the golden memory model.
- **Agent:** A new `controller_agent` class encapsulates the driver, sequencer, and monitor, providing a cohesive interface parallel to each master interface.
- **Environment:** The `controller_env` class groups agents and the scoreboard, managing the overall testbench infrastructure.
- **Test:** The original `initial` block is replaced by `controller_test`, which orchestrates the test flow through UVM phases (`build_phase`, `connect_phase`, `run_phase`).

3.2 Component Details

3.2.1 controller_transaction

Encapsulates a dual-master transaction with the following properties:

- Master 0 and Master 1 fields: address, write data, write enable and request flags
- Randomizable delay between transactions
- Constraint limiting delay to 0-8 cycles
- String representation for logging via `convert2string()`

3.2.2 controller_sequencer

A parametrized sequencer managing the flow of `controller_transaction` objects from sequences to the driver. Implements the standard UVM pull interface.

3.2.3 controller_driver

Pulls transactions from the sequencer and applies them to the DUT:

- Drives request signals (`req0`, `req1`)
- Applies address and data based on transaction fields
- Handles clock-based handshaking with the DUT
- Respects grant signals (`gnt0`, `gnt1`)

3.2.4 controller_monitor

Passively observes DUT transactions:

- Samples grant and control signals
- Captures read and write operations
- Creates transaction objects representing observed bus activity
- Broadcasts via TLM analysis port to scoreboard and functional coverage

3.2.5 controller_scoreboard

Maintains a golden memory model and validates correctness:

- Receives predicted transactions from driver (via sequencer)
- Receives observed transactions from monitor
- Compares actual versus predicted reads
- Reports failures and accumulates statistics

3.2.6 controller_agent

Hierarchical component grouping verification components:

- Contains: driver, sequencer, monitor
- Provides configuration options (active vs. passive)
- Handles instantiation and connection of sub-components

3.2.7 **controller_env**

Top-level environment containing:

- One or more `controller_agent` instances (one per master, or combined)
- `controller_scoreboard` for result checking
- Manages build and connect phases

3.2.8 **controller_sequence**

Generates sequences of transactions with various test scenarios:

- Deterministic writes and reads, specific to the master
- DMA access patterns
- Contention scenarios (both masters requesting simultaneously)
- Randomized mixed operations

3.2.9 **controller_test**

Orchestrates the test execution:

- Configures environment and test parameters
- Instantiates sequences
- Controls test phases and termination

4 UVM Architecture Diagram

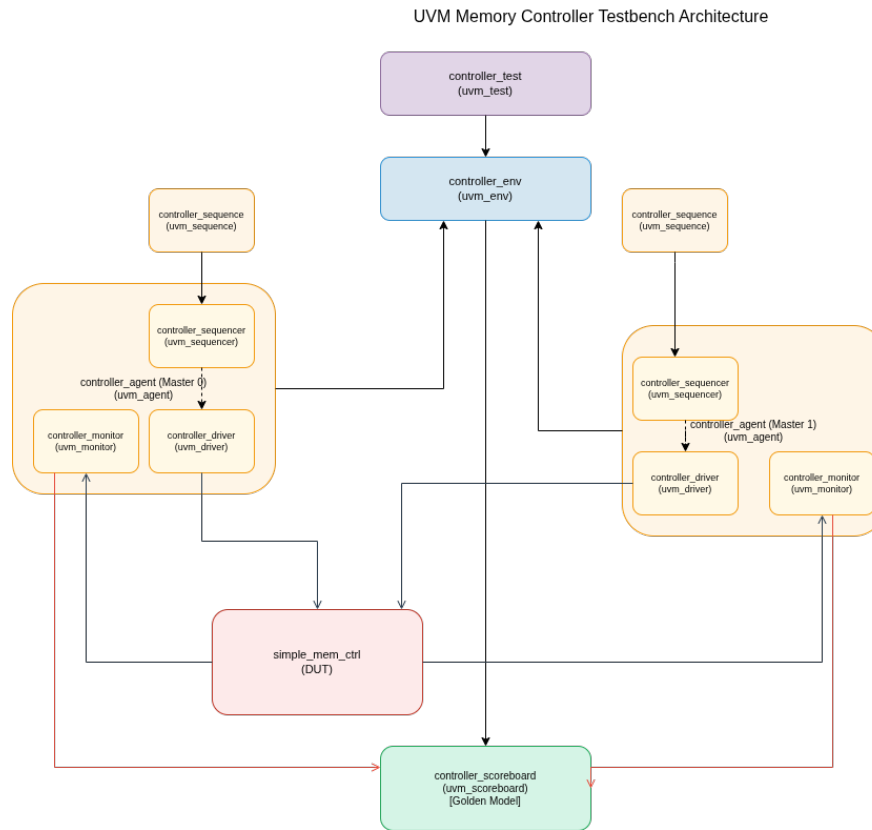


Figure 1: UVM Testbench Architecture for Memory Controller Verification

5 UVM Code

```

`timescale 1ns/1ps
`include "uvm_macros.svh"

package controller_pkg;
    import uvm_pkg::*;

    parameter ADDR_WIDTH = 8;
    parameter DATA_WIDTH = 32;

    class controller_transaction extends uvm_sequence_item;
    `uvm_object_utils(controller_transaction)

        // Master 0
        random logic [ADDR_WIDTH-1:0]    addr0;
        random logic [DATA_WIDTH-1:0]    wdata0;
        random logic                      is_write0;
        random logic                      use_m0;

        random logic [ADDR_WIDTH-1:0]    addr1;
    
```

```

    random logic [DATA_WIDTH-1:0]    wdata1;
    random logic                      is_write1;
    random logic                      use_m1;

    random int delay_cycles;
    random bit is_end;

    constraint valid_delay { delay_cycles inside {[0:8]}; }

    function new(string name = "controller_transaction");
        super.new(name);
    endfunction

    function string convert2string();
    return $sformatf("M0: use=%b addr=%0d we=%b | M1: use=%b addr=%0d we=%b | delay=%0d",
                    use_m0, addr0, is_write0, use_m1, addr1, is_write1, delay_cycles);
    endfunction

endclass : controller_transaction

class controller_sequencer extends uvm_sequencer #(controller_transaction);
`uvm_object_utils(controller_sequencer)

    function new(string name = "controller_sequencer");
        super.new(name);
    endfunction

endclass

class controller_driver extends uvm_driver #(controller_transaction);
`uvm_component_utils(controller_driver)
endclass : controller_driver

class controller_monitor extends uvm_monitor;
`uvm_component_utils(controller_monitor)
endclass : controller_monitor

class controller_agent extends uvm_agent;
`uvm_component_utils(controller_agent)
endclass : controller_agent

class controller_scoreboard extends uvm_scoreboard;
`uvm_component_utils(controller_scoreboard)
endclass : controller_scoreboard

class controller_en extends uvm_env;
`uvm_component_utils(controller_en)
endclass

class controller_sequence extends uvm_sequence #(controller_transaction);
`uvm_object_utils(controller_sequence)
endclass : controller_sequence

class controller_test extends uvm_test;
`uvm_component_utils(controller_test)

```

```
        endclass: controller_test

endpackage : controller_pkg

module tb_uvm_simple_mem_ctrl;
    import uvm_pkg::*;
    import controller_pkg::*;
    // Parameters must match DUT
    parameter ADDR_WIDTH = 8;
    parameter DATA_WIDTH = 32;
endmodule : tb_uvm_simple_mem_ctrl
```

6 Conclusion

7 Appendix

7.1 Number of hours required

For the completion of this laboratory, it has been needed x hours.

7.2 Appendix A